LEVEL 1 – BASIC

A. HTML FUNDAMENTALS

---

1. What is the purpose of <!DOCTYPE html>?

<!DOCTYPE html> declares that the document is written in HTML5. It tells the browser to render the page in Standards Mode instead of Quirks Mode, ensuring consistent layout and behavior across browsers.

---

2. Explain the basic structure of an HTML document.

An HTML document starts with <!DOCTYPE html>, followed by the <html> root element. Inside <html>, there are two main sections: <head> (metadata, title, CSS, scripts) and <body> (visible content shown to users).

---

3. What is the difference between <head> and <body>?

The <head> contains metadata such as title, meta tags, CSS links, and scripts. The <body> contains all visible content like text, images, forms, tables, and buttons.

---

4. What is HTML5 and how is it different from earlier versions?

HTML5 is the latest version of HTML that introduced semantic elements, multimedia support (audio/video without plugins), canvas for graphics, local storage, and improved form input types. Earlier versions required plugins and lacked semantic structure.

---

5. What are semantic elements? Give examples.

Semantic elements clearly describe the meaning of their content. Examples include <header>, <footer>, <section>, <article>, <nav>, and <main>. They improve SEO, readability, and accessibility.

---

6. What is the difference between and ?

 is a block-level element that takes full width and starts on a new line. is an inline element that takes only the width of its content and stays in the same line.

---

7. What are block-level and inline elements?

Block-level elements start on a new line and take full width (e.g., , <p>). Inline elements stay in the same line and take only content width (e.g., , <a>).

---

8. What is the purpose of the <meta> tag?

The <meta> tag provides metadata such as character encoding, viewport settings, author name, and SEO description. It helps browsers and search engines interpret the page correctly.

---

9. What is <br> tag?

 is used to insert a line break. It moves content to the next line without creating a new paragraph.

---

10. What is <title> tag?

<title> defines the webpage title shown in the browser tab. It is important for SEO and bookmarking.

---

11. How do you make an email link?

An email link is created using <a href="mailto:example@gmail.com">Email</a>. Clicking it opens the default email application.

---

12. Explain the difference between tag and element.

A tag is markup enclosed in angle brackets like <p>. An element includes the opening tag, content, and closing tag together.

---

13. What is <img> tag used for?

<img> is used to embed images into a webpage. It requires the src attribute to define the image path.

---

14. What is the alt attribute in <img>?

The alt attribute provides alternative text if the image fails to load. It improves accessibility and helps search engines understand the image.

---

15. What is an HTML attribute? Give examples.

An attribute provides extra information about an element and is written inside the opening tag. Examples include id, class, src, href, and style.

---

16. What are lists? Difference between <ul> and <ol>?

Lists group related items together. <ul> creates an unordered (bulleted) list, while <ol> creates an ordered (numbered) list.

---

17. How do you make a table?

A table is created using <table>, with rows defined by <tr> and cells defined by <td> (data) or <th> (header).

---

18. What is the difference between <td> and <th>?

<td> is a normal data cell. <th> is a header cell and is bold and centered by default.

---

19. What are colspan and rowspan?

colspan merges multiple columns into one cell. rowspan merges multiple rows into one cell.

---

20. What are Semantic and Non-Semantic elements? Give examples.

Semantic elements describe meaning (e.g., <section>). Non-semantic elements like and do not describe content meaning.

---

21. When do you use Ordered list and Unordered list?

Use <ol> when sequence matters, such as steps. Use <ul> when order does not matter.

---

22. What is the key feature of <tfoot> in HTML table?

<tfoot> groups footer content in a table and is rendered at the bottom, even if written before <tbody>.

---

23. What is <base> tag?

<base> sets a base URL for all relative links in the document.

---

24. How do you make a hidden input field?

Use <input type="hidden" name="key" value="value">. It stores data without displaying it.

---

25. What is child tag?

A child tag is an element nested inside another element (parent). This relationship forms the DOM structure.

B. NAVIGATION & FORMS

---

1. What is the purpose of <a> tag?

The <a> (anchor) tag is used to create hyperlinks that allow users to navigate between pages, sections, or external websites. It uses the href attribute to define the destination.

---

2. How do you create a hyperlink?

A hyperlink is created using <a href="URL">Link Text</a>.
The href attribute specifies the URL of the target page.

---

3. Difference between absolute and relative paths?

An absolute path includes the full URL including protocol and domain (e.g., https://example.com/page.html).
A relative path specifies the file location relative to the current file (e.g., ../images/pic.jpg).

---

4. How do you open a link in a new tab?

Use the target="_blank" attribute inside the anchor tag.
Example: <a href="URL" target="_blank">Open</a>.

---

5. What is <form> used for?

The <form> element collects user input and sends it to a server.
It contains input fields like text boxes, radio buttons, checkboxes, and submit buttons.

---

6. Difference between GET and POST?

GET sends data in the URL and is visible in the address bar.
POST sends data in the request body and is more secure for sensitive information.

---

7. What is required attribute?

The required attribute ensures that a form field must be filled before submission.
It prevents empty form submission on the client side.

---

8. Difference between radio button and checkbox? How do you make checkbox?

Radio buttons allow only one option selection in a group.
Checkboxes allow multiple selections and are created using <input type="checkbox">.

---

9. What is <label> used for?

The <label> tag connects descriptive text to an input field.
Clicking the label focuses or selects the associated input.

---

10. How do you make dropdown list?

A dropdown is created using <select> and <option> elements.
Users can choose one option from the list.

---

11. What is <datalist>? What is it used for?

<datalist> provides predefined suggestions for an input field.
It allows users to either select from suggestions or type their own value.

---

12. What is <progress> tag?

<progress> represents task completion progress.
It visually shows how much of a task is completed.

---

13. What is <meter> tag and how is it different from <progress>?

<meter> represents a value within a fixed range (like battery level).
<progress> represents task completion over time.

---

14. How do you reset a form?

Use <input type="reset"> inside the form.
It clears all input fields and restores default values.

---

15. Explain different attributes and usage of <a> tag in HTML.

Common attributes include href (destination URL), target (where to open), title (tooltip text), and download (download file).
It is mainly used for navigation and linking resources.

---

16. What are the various input types available in HTML?

Common types include text, password, email, number, radio, checkbox, date, file, range, submit, reset, and hidden.
Each type is designed for specific user input purposes.

C. CSS BASICS

---

1. What is CSS?

CSS (Cascading Style Sheets) is used to style HTML elements.
It controls layout, colors, fonts, spacing, positioning, and responsiveness.

---

2. Different ways to apply CSS?

CSS can be applied using inline styles, internal styles inside <style> tag, or external .css files.
External CSS is preferred for maintainability.

---

3. What is a selector? What are the different types of CSS selectors?

A selector targets HTML elements to apply styles.
Types include element selector, class selector, id selector, attribute selector, and pseudo-classes.

---

4. Difference between class and id?

An id is unique and used only once per page.
A class can be reused for multiple elements.

---

5. What is box model?

The box model consists of content, padding, border, and margin.
It defines how the total size of an element is calculated.

---

6. Difference between margin and padding?

Margin is the outer space outside the border.
Padding is the inner space between content and border.

---

7. What is display: block vs inline?

block elements take full width and start on a new line.
inline elements take only content width and stay in the same line.

---

8. What is responsive design?

Responsive design ensures webpages adapt to different screen sizes.
It uses flexible layouts and media queries.

---

9. What is a media query?

A media query applies CSS rules based on device characteristics like screen width.
It is used to build responsive designs.

---

10. What is box-sizing?

box-sizing defines how width and height are calculated.
border-box includes padding and border inside the defined width.

---

11. What is the difference between inline, block, and inline-block?

Inline takes content width and cannot set width/height.
Block takes full width; inline-block allows width/height but stays inline.

---

12. What is inheritance in CSS?

Inheritance allows child elements to inherit certain properties from parent elements.
Properties like color and font-family are commonly inherited.

D. JAVASCRIPT BASICS

---

1. What is JavaScript?

JavaScript is a scripting language used to make webpages interactive.
It allows dynamic content updates and user interaction handling.

---

2. How do you include JavaScript in an HTML page?

JavaScript is included using the <script> tag.
It can be written internally or linked as an external file.

---

3. Difference between var, let, const?

var has function scope and allows redeclaration.
let and const have block scope; const cannot be reassigned.

---

4. What are data types in JavaScript?

Primitive types include string, number, boolean, null, undefined, symbol, and bigint.
Non-primitive type is object.

---

5. What is DOM?

DOM (Document Object Model) represents HTML as a tree structure.
JavaScript uses DOM to access and modify elements.

---

6. What is an event?

An event is a user interaction such as click, submit, or keypress.
JavaScript can respond to these events.

---

7. Difference between innerHTML and textContent?

innerHTML reads or writes HTML content including tags.
textContent reads or writes only plain text.

---

8. What is console.log() used for?

console.log() prints messages in the browser console.
It is mainly used for debugging.

---

9. Difference between == vs ===?

== compares values after type conversion.
=== compares both value and type strictly.

---

10. What is null vs undefined?

null represents intentional absence of value.
undefined means a variable is declared but not assigned.

---

11. What is scope?

Scope determines where a variable can be accessed.
It can be global, function, or block scope.

---

12. What is block scope?

Block scope means variables declared with let or const inside {} are accessible only within that block.

---

13. What is control flow?

Control flow determines the execution order of code.
Examples include if statements, loops, and switch statements.

---

14. What are different ways an HTML element can be accessed in JavaScript?

Elements can be accessed using getElementById, getElementsByClassName, querySelector, and querySelectorAll.

LEVEL 2 – INTERMEDIATE (Application & Reasoning)

HTML & FORMS

---

1. Why should tables not be used for layout?

Tables are meant for tabular data, not page layout.
Using tables for layout reduces flexibility, responsiveness, and makes maintenance difficult.

---

2. What happens if two elements have the same id?

IDs must be unique in a document.
If duplicated, JavaScript may only select the first match and CSS behavior becomes unpredictable.

---

3. Why is semantic HTML important for accessibility?

Semantic HTML helps screen readers understand content structure.
It allows assistive technologies to interpret headings, navigation, and sections correctly.

---

4. What happens if form method is GET and we send password?

The password appears in the URL.
This is insecure because URLs can be stored in browser history and server logs.

---

5. Difference between id and name in forms?

id is used for CSS styling and JavaScript selection.
name is used to send data to the server during form submission.

---

6. Difference between <select> and <datalist>?

<select> restricts users to predefined options only.
<datalist> provides suggestions but allows custom input.

---

7. How does browser resolve relative paths?

The browser resolves relative paths based on the current document's location.
It calculates the path relative to the current URL.

---

8. What happens if <!DOCTYPE html> is missing?

The browser enters Quirks Mode.
This may cause inconsistent rendering and layout issues.

---

9. What happens if the viewport meta tag is not added?

The page may not scale properly on mobile devices.
It can appear zoomed out or improperly sized.

CSS – INTERMEDIATE

---

1. What is CSS specificity? How is it calculated?

Specificity determines which CSS rule applies when multiple rules target the same element.
Priority order: Inline styles > ID selectors > Class selectors > Element selectors.

---

2. Which has more priority: inline, id, class?

Inline styles have highest priority.
Then id selectors, then class selectors.

---

3. What is !important? Why should it be avoided?

!important overrides all other styles regardless of specificity.
It should be avoided because it makes CSS hard to maintain and debug.

---

4. Difference between display: none and visibility: hidden?

display: none removes the element from layout completely.
visibility: hidden hides the element but keeps its space reserved.

---

5. Explain position: relative, absolute, fixed, sticky.

Relative positions element relative to itself.
Absolute positions relative to nearest positioned ancestor.
Fixed positions relative to viewport.
Sticky toggles between relative and fixed during scrolling.

---

6. What happens when padding increases total width?

If box-sizing is content-box, padding increases total element width.
If box-sizing is border-box, padding is included inside defined width.

---

7. What is overflow property?

The overflow property controls what happens when content exceeds container size.
Values include visible, hidden, scroll, and auto.

---

8. Explain about layout?

Layout refers to how elements are arranged on a page.
Modern layouts use Flexbox and Grid instead of floats or tables.

---

9. Difference between flexbox and grid?

Flexbox is one-dimensional (row or column).
Grid is two-dimensional (rows and columns).

---

10. What is z-index?

z-index controls stacking order of positioned elements.
Higher z-index appears on top of lower ones.

---

11. What happens when two selectors have equal specificity?

The last declared rule in the CSS file will apply.
Order of appearance matters.

---

12. What determines the containing block for absolute positioning?

The nearest positioned ancestor (with position relative, absolute, or fixed).
If none exists, it uses the viewport.

JAVASCRIPT – CORE (INTERMEDIATE)

---

1. What is hoisting?

Hoisting is JavaScript behavior where variable and function declarations are moved to the top of their scope before execution.

---

2. Difference between function declaration and function expression?

Function declarations are hoisted completely.
Function expressions are not fully hoisted and cannot be called before definition.

---

3. What is an arrow function?

An arrow function is a shorter syntax for writing functions.
It does not have its own this context.

---

4. What is type coercion?

Type coercion is automatic conversion of one data type to another.
It commonly happens in comparisons using ==.

---

5. What is event bubbling?

Event bubbling means an event triggered on a child element propagates upward to its parent elements.

---

6. What is event delegation?

Event delegation attaches a single event listener to a parent element.
It handles events for multiple child elements efficiently.

---

7. What happens if JS is written in <head> before DOM loads?

JavaScript may try to access elements before they exist.
This can result in null errors unless DOMContentLoaded is used.

---

8. What is closure? (basic explanation)

A closure is a function that remembers variables from its outer scope.
It retains access even after the outer function finishes execution.

---

9. Difference between primitive and reference types?

Primitive types store actual values.
Reference types store memory addresses pointing to objects.

---

10. What is the difference between global scope and local scope?

Global scope variables are accessible everywhere.
Local scope variables are accessible only inside a function or block.

---

11. Why does getElementById() sometimes return null?

It returns null if the element does not exist.
It can also return null if script runs before DOM loads.

LEVEL 3 – CROSS QUESTIONS

HTML – DEEP PROBING

---

1. You say semantic HTML improves SEO. How exactly? What does search engine read?

Search engines read the HTML structure of a webpage to understand content hierarchy. Semantic tags like <header>, <section>, <article>, and <nav> help search engines identify headings, main content, and navigation areas, improving indexing and ranking.

---

2. If <section> and look same visually, why use <section>?

<section> provides semantic meaning, indicating a thematic grouping of content.
 is purely structural with no meaning; search engines and assistive technologies cannot infer context from it.

---

3. What happens if we remove <!DOCTYPE html>?

The browser switches to Quirks Mode.
This causes older rendering behavior and may break layouts or cause inconsistent styling across browsers.

---

4. Why is alt important beyond accessibility?

The alt attribute helps search engines understand image content.
It also displays fallback text when images fail to load, improving usability and SEO.

---

5. If two elements share same id, how will JavaScript behave?

JavaScript functions like getElementById() will return only the first matching element.
This creates unpredictable behavior and breaks proper DOM manipulation.

---

6. How does browser render HTML from top to bottom?

The browser parses HTML sequentially and builds the DOM tree.
It then applies CSS to create the render tree and paints elements onto the screen.

CSS – CROSS QUESTIONS

---

1. Two selectors:

#id .class p
div p
Which has higher specificity and why?

#id .class p has higher specificity because it includes an ID selector.
ID selectors carry more weight than element selectors like div p.

---

2. Why is layout breaking when width + padding exceed container?

By default, width does not include padding.
If padding increases total size beyond container width, it causes overflow unless box-sizing: border-box is used.

---

3. If absolute element moves unexpectedly, what determines its position?

An absolutely positioned element is positioned relative to the nearest positioned ancestor.
If no positioned ancestor exists, it is positioned relative to the viewport.

---

4. When would you use flexbox over grid?

Use Flexbox when aligning items in a single direction (row or column).
Use Grid when designing complex two-dimensional layouts with rows and columns.

---

5. Why should !important not be used in large projects?

!important overrides normal specificity rules and makes debugging difficult.
Overuse leads to messy, hard-to-maintain CSS.

---

6. If inline style and id selector conflict, which wins and why?

Inline styles win because they have higher specificity than ID selectors.
Inline styles are considered the highest priority in normal CSS hierarchy.

---

7. Why is vertical margin not applied properly on inline elements?

Inline elements do not respect vertical margin the same way block elements do.
To apply vertical margin properly, change the element to display: block or inline-block.

JAVASCRIPT – DEEP PROBING

---

1. Why does this print 5 five times?

```
for (var i = 0; i < 5; i++) {
  setTimeout(() => console.log(i), 1000);
}
```

Because var is function-scoped, all callbacks share the same i.
By the time setTimeout runs, the loop has completed and i equals 5.

Follow-up: How to fix?

Replace var with let.
let is block-scoped, so each iteration gets its own copy of i.

Follow-up: Why does let fix it?

Because let creates a new binding for each loop iteration.
Each callback remembers its own i value.

---

2. What is difference between:

function test() {}
const test = function() {}

Function declarations are hoisted completely and can be called before definition.
Function expressions are not fully hoisted and cannot be called before assignment.

---

3. Why is using innerHTML with user input dangerous?

It can allow malicious scripts to be injected into the page.
This leads to Cross-Site Scripting (XSS) attacks.

Follow-up: What attack is this?

This is called XSS (Cross-Site Scripting).
Attackers inject scripts that run in other users' browsers.

Follow-up: How to prevent it?

Use textContent instead of innerHTML.
Sanitize and validate all user input before inserting into DOM.

---

4. Explain event bubbling with real example.

If you click a button inside a div, the click event triggers on the button first.
Then it propagates upward to the parent div and then to the document.

Follow-up: How to stop it?

Use event.stopPropagation() inside the event handler.
This prevents the event from moving to parent elements.

---

5. If page is not responsive on mobile, what might be missing?

The viewport meta tag may be missing.
Without it, the page will not scale correctly on mobile devices.

Follow-up: What does viewport meta tag do?

It controls page scaling and width on different devices.
Example: <meta name="viewport" content="width=device-width, initial-scale=1.0">

---

6. Explain scope chain with example.

When a variable is accessed, JavaScript first checks local scope.
If not found, it searches outer scopes until reaching global scope.

---

7. What will this output and why?

```
console.log(a);
var a = 10;
```

It prints undefined.
Because var is hoisted but initialized as undefined before assignment.

---

8. What happens if you declare a variable without var/let/const?

It becomes a global variable automatically.
This can cause unintended side effects and is considered bad practice.

LOGIC QUESTIONS

---

1. Reverse a string.

Convert the string to an array using split(""), reverse it using reverse(), and join it back using join("").

---

2. Check if a string is palindrome.

Reverse the string and compare it with the original.
If both are equal, it is a palindrome.

---

3. Find largest number in array.

Use a loop to compare values or use Math.max(...array) to find the maximum.

---

4. Count vowels in string.

Loop through the string and count characters that match a, e, i, o, u.

---

5. Remove duplicates from array.

Use new Set(array) to remove duplicates.
Convert back to array if needed.

---

6. Write logic to check if a number is even or odd.

If number % 2 === 0, it is even.
Otherwise, it is odd.

---

7. Write a loop to print numbers from 1 to 10.

Use a for loop:
for (let i = 1; i <= 10; i++) { console.log(i); }

ADDITIONAL QUESTIONS

---

1. Explain responsive design to a non-technical client.

Responsive design means the website automatically adjusts to different screen sizes.
It ensures the site looks good on mobile, tablet, and desktop.

---

2. Explain why form validation is important from business perspective.

Form validation ensures correct and complete data submission.
It reduces errors, improves user experience, and prevents loss of business information.

---

3. If client says page is slow, what will you check first?

Check large images, unnecessary scripts, and excessive DOM manipulation.
Also inspect network requests and browser performance tools.

---

4. How will you debug if CSS is not applying?

Check selector correctness, spelling errors, specificity conflicts, and file linking.
Use browser DevTools to inspect applied styles.

---

5. How do you structure CSS in large project?

Use modular CSS files and consistent naming conventions like BEM.
Separate layout, components, and utilities for maintainability.

---

6. If JavaScript is not working on production but works locally, what could be reasons?

Possible reasons include incorrect file paths, server configuration issues, caching problems, or minification errors.